# Les boucles en Swift

*420-249-AH - Programmation mobile iOS*

## 1. Pourquoi utiliser une boucle ?

Dans une application, certaines opérations doivent être répétées plusieurs fois.

Exemples :

- Vérifier plusieurs tentatives de connexion
- Répéter une opération jusqu'à ce qu'elle réussisse
- Afficher une progression
- Effectuer plusieurs étapes d'un traitement
- Parcourir une série de valeurs
- Répéter une action tant qu'une condition est vraie

Sans boucle, il faudrait répéter manuellement les mêmes instructions.

Exemple :

```
print("Tentative 1")
print("Tentative 2")
print("Tentative 3")
print("Tentative 4")
print("Tentative 5")
```

Avec une boucle :

```
for tentative in 1...5 {
    print("Tentative \(tentative)")
}
```

## 2. Les principales boucles en Swift

Swift propose principalement :

```
for-in
while
repeat-while
```

Chaque boucle répond à un besoin différent.

| Boucle | Utilisation |
|---|---|
| `for-in` | Lorsque le nombre d'itérations est connu, ou pour parcourir une séquence |
| `while` | Répéter tant qu'une condition est vraie |
| `repeat-while` | Comme while, mais exécute le bloc au moins une fois |

## 3. La boucle for-in

La boucle for-in est probablement la boucle la plus utilisée en Swift.

Syntaxe :

```
for variable in sequence {
    // instructions
}
```

Exemple :

```
for numero in 1...5 {
    print(numero)
}
```

Résultat :

```
Console
1
2
3
4
5
```

À chaque itération, numero reçoit une nouvelle valeur.

## 4. Comprendre l'exécution d'un for

Avec :

```
for numero in 1...3 {
    print("Écran \(numero)")
}
```

Swift effectue :

```
numero = 1  →  Écran 1
numero = 2  →  Écran 2
numero = 3  →  Écran 3
```

Résultat :

```
Console
Écran 1
Écran 2
Écran 3
```

## 5. Les intervalles fermé en Swift

```
1...5
```

signifie :

```
1, 2, 3, 4, 5
```

Les deux limites sont incluses.

Exemple :

```
for numero in 1...5 {
    print(numero)
}
```

## 6. L'intervalle semi-ouvert en Swift

Swift permet également d'utiliser :

```
1..<5
```

Résultat :

```
1, 2, 3, 4
```

La limite supérieure n'est pas incluse.

Comparaison :

| Syntaxe | Résultat |
|---|---|
| 1...5 | 1 2 3 4 5 |
| 1..<5 | 1 2 3 4 |

## 7. Pourquoi ..< est-il utile ?

En programmation mobile, on travaille souvent avec des positions commençant à 0.

Exemple conceptuel :

```
Page 0
Page 1
Page 2
Page 3
```

Pour quatre pages :

```
for index in 0..<4 {
    print("Page \(index)")
}
```

Résultat :

```
Console
Page 0
Page 1
Page 2
Page 3
```

## 8. Exemple — Étapes d'installation

Supposons qu'une application effectue cinq étapes lors de son initialisation.

```swift
for etape in 1...5 {
    print("Initialisation - étape \(etape)/5")
}
```

Résultat :

```
Console
Initialisation - étape 1/5
Initialisation - étape 2/5
Initialisation - étape 3/5
Initialisation - étape 4/5
Initialisation - étape 5/5
```

## 9. Exemple — Simulation d'une progression

On peut simuler une progression :

```swift
for progression in 1...10 {
    let pourcentage = progression * 10
    print("Téléchargement : \(pourcentage)%")
}
```

Résultat :

```
Console
Téléchargement : 10%
Téléchargement : 20%
Téléchargement : 30%
...
Téléchargement : 100%
```

Ici, la boucle contrôle le nombre d'itérations et le calcul du pourcentage.

## 10. Lorsque la variable de boucle n'est pas nécessaire

Parfois, on souhaite simplement répéter une instruction.

Exemple :

```
for _ in 1...3 {
    print("Nouvelle tentative...")
}
```

Résultat :

```
Console
Nouvelle tentative...
Nouvelle tentative...
Nouvelle tentative...
```

Le symbole _ signifie ici : la valeur courante ne nous intéresse pas.

## 11. Compter avec un pas différent

Supposons que l'on souhaite afficher une progression :

```
0   20   40   60   80   100
```

Un intervalle classique avance généralement de 1. Swift fournit :

```
stride(...)
```

Exemple :

```
for progression in stride(from: 0, through: 100, by: 20) {
    print("\(progression)%")
}
```

Résultat :

```
Console
0%
20%
40%
60%
80%
100%
```

## 12. through et to

Avec :

```
stride(from: 0, through: 100, by: 20)
```

100 est inclus.

```
0  20  40  60  80  100
```

Avec :

```
stride(from: 0, to: 100, by: 20)
```

100 est exclu.

```
0  20  40  60  80
```

On retrouve donc une logique similaire à celle de ... et ..<.

## 13. Compter à rebours

stride permet également de diminuer une valeur.

Exemple :

```
for secondes in stride(from: 5, through: 1, by: -1) {
    print(secondes)
}
print("Démarrage")
```

Résultat :

```
Console
5
4
3
2
1
Démarrage
```

Exemples de contexte mobile :

- Compte à rebours avant une opération
- Minuterie
- Délai de reconnexion
- Expiration temporaire d'une action

## 14. La boucle while

while répète un bloc tant qu'une condition est vraie.

Syntaxe :

```
while condition {
    // instructions
}
```

Exemple :

```
var compteur = 1
while compteur <= 5 {
    print(compteur)
    compteur += 1
}
```

## 15. Comprendre while

Avec :

```
var tentative = 1
while tentative <= 3 {
    print("Tentative \(tentative)")
    tentative += 1
}
```

Fonctionnement :

```
tentative <= 3 ?
      | Oui
      ▼
Exécuter le bloc
      |
      ▼
tentative += 1
      |
      ▼
Tester de nouveau
```

Lorsque la condition devient fausse, la boucle s'arrête.

## 16. Attention aux boucles infinies

Considérons :

```
var compteur = 1
while compteur <= 5 {
    print(compteur)
}
```

Problème : compteur ne change jamais. La condition compteur <= 5 reste donc toujours vraie.

Résultat :

```
Console
1
1
1
1
1
1
...
```

**Attention :** c'est une boucle infinie.

## 17. Éviter une boucle infinie

Il faut généralement s'assurer qu'une donnée utilisée dans la condition évolue.

Correct :

```
var compteur = 1
while compteur <= 5 {
    print(compteur)
    compteur += 1
}
```

À chaque passage :

```
1 → 2 → 3 → 4 → 5 → 6
```

Quand compteur = 6, la condition devient 6 <= 5 → false. La boucle s'arrête.

## 18. Exemple — Simulation des tentatives de connexion

Une application peut limiter le nombre de tentatives.

```
var tentative = 1
let maximumTentatives = 3
while tentative <= maximumTentatives {
    print("Tentative de connexion \(tentative)")
    tentative += 1
}
print("Fin des tentatives")
```

Résultat :

```
Console
Tentative de connexion 1
Tentative de connexion 2
Tentative de connexion 3
Fin des tentatives
```

## 19. Exemple — Niveau de batterie simulé

```swift
var batterie = 100
while batterie > 0 {
    print("Batterie : \(batterie)%")
    batterie -= 20
}
print("Batterie épuisée")
```

Résultat :

```
Console
Batterie : 100%
Batterie : 80%
Batterie : 60%
Batterie : 40%
Batterie : 20%
Batterie épuisée
```

## 20. La boucle repeat-while

repeat-while ressemble à while.

Syntaxe :

```swift
repeat {
    // instructions
} while condition
```

Différence importante : le bloc est exécuté avant de vérifier la condition.

> **À retenir :** il sera exécuté au moins une fois.

## 21. while vs repeat-while

Avec while :

```swift
var nombre = 10
while nombre < 5 {
    print(nombre)
}
```

Le bloc n'est jamais exécuté, car 10 < 5 → false.

Avec repeat-while :

```swift
var nombre = 10
repeat {
    print(nombre)
} while nombre < 5
```

Résultat :

```
Console
10
```

Le bloc est exécuté une première fois avant le test.

## 22. Exemple — Demande d'un code d'accès

Supposons que l'on simule la saisie d'un code d'accès.

```swift
var codeCorrect = false
var tentative = 0
repeat {
    tentative += 1
    print("Vérification du code - tentative \(tentative)")
    if tentative == 3 {
        codeCorrect = true
    }
} while !codeCorrect
print("Accès autorisé")
```

Ici, la vérification doit être effectuée au moins une fois : repeat-while est donc approprié.

## 23. Choisir entre for et while

Utiliser généralement for-in lorsque le nombre d'itérations est connu.

```swift
for tentative in 1...3 {
    print(tentative)
}
```

On sait qu'il y aura 3 itérations.

Utiliser généralement while lorsque le nombre d'itérations dépend d'une condition.

```swift
while !connexionReussie {
    // nouvelle tentative
}
```

On ne sait pas nécessairement à l'avance combien d'itérations seront nécessaires.

## 24. Choisir entre while et repeat-while

| Boucle | Comportement |
|---|---|
| `while` | La condition est vérifiée avant l'exécution (tester → exécuter) |
| `repeat-while` | La condition est vérifiée après l'exécution (exécuter → tester) |

Donc : repeat-while garantit au moins une exécution.

## 25. Arrêter une boucle avec break

break permet de quitter immédiatement une boucle.

Exemple :

```
for tentative in 1...5 {
    print("Tentative \(tentative)")
    if tentative == 3 {
        print("Connexion réussie")
        break
    }
}
```

Résultat :

```
Console
Tentative 1
Tentative 2
Tentative 3
Connexion réussie
```

Les tentatives 4 et 5 ne sont jamais exécutées.

## 26. Exemple — Recherche d'une connexion

```
var connexionReussie = false
for tentative in 1...5 {
    print("Connexion \(tentative)...")
    if tentative == 3 {
        connexionReussie = true
        print("Connexion établie")
        break
    }
}
```

Dès que la connexion réussit, break arrête les tentatives restantes.

## 27. Ignorer une itération avec continue

continue ne termine pas complètement la boucle. Il passe directement à l'itération suivante.

Exemple :

```
for numero in 1...5 {
    if numero == 3 {
        continue
    }
    print(numero)
}
```

Résultat :

```
Console
1
2
4
5
```

La valeur 3 a été ignorée.

## 28. Exemple — Ignorer une étape

Supposons qu'une étape soit temporairement désactivée :

```swift
for etape in 1...5 {
    if etape == 3 {
        print("Étape 3 ignorée")
        continue
    }
    print("Traitement de l'étape \(etape)")
}
```

Résultat :

```
Console
Traitement de l'étape 1
Traitement de l'étape 2
Étape 3 ignorée
Traitement de l'étape 4
Traitement de l'étape 5
```

## 29. break et continue

Il ne faut pas les confondre.

| Instruction | Effet |
|---|---|
| `break` | Quitte complètement la boucle |
| `continue` | Ignore seulement l'itération actuelle et poursuit avec la suivante |

Exemple conceptuel — avec continue :

```
1  2  3 ← continue  4  5
La boucle continue.
```

Avec break :

```
1  2  3 ← break
La boucle est terminée.
```

## 30. Combiner boucles et conditions

Les boucles deviennent particulièrement utiles lorsqu'elles sont combinées avec les conditions.

Exemple :

```
for niveau in 1...10 {
    if niveau < 4 {
        print("Niveau \(niveau) : débutant")
    } else if niveau < 8 {
        print("Niveau \(niveau) : intermédiaire")
    } else {
        print("Niveau \(niveau) : avancé")
    }
}
```

On combine ici : for + if + else if + else.

## 31. Exemple - Tentatives de code PIN

Une application permet un maximum de trois tentatives.

```
let codeCorrect = 1234
var codeSaisi = 1111
for tentative in 1...3 {
    print("Tentative \(tentative)")
    if codeSaisi == codeCorrect {
        print("Code correct")
        break
    } else {
        print("Code incorrect")
    }
    codeSaisi += 123
}
```

L'objectif ici est d'observer la combinaison : boucle + condition + variable + break.

## 32. Exemple - Simulation d'une synchronisation

On souhaite afficher la progression d'une synchronisation mobile.

```
for progression in stride(from: 0, through: 100, by: 10) {
    print("Synchronisation : \(progression)%")
    if progression == 100 {
        print("Synchronisation terminée")
    }
}
```

Résultat :

```
Console
Synchronisation : 0%
Synchronisation : 10%
Synchronisation : 20%
...
Synchronisation : 100%
Synchronisation terminée
```

## 33. Exercice - Simulation d'un téléchargement

Créer un programme Swift simulant le téléchargement d'un fichier.

Le téléchargement commence à 0 % et progresse de 10 % jusqu'à 100 %.

À chaque étape, afficher :

```
Console
Téléchargement : 10 %
Téléchargement : 20 %
Téléchargement : 30 %
...
```

Lorsque la progression atteint 50 %, afficher également :

```
Console
La moitié du téléchargement est terminée.
```

À 100 %, afficher :

```
Console
Téléchargement terminé.
```

**Contraintes — utiliser :**

- Une boucle
- Une condition
- Une variable ou constante
- L'interpolation de chaînes

## 34. Exercice - Tentatives de connexion

Une application permet un maximum de 4 tentatives.

Créer :

```swift
let maximumTentatives = 4
```

À chaque tentative, afficher :

```
Console
Tentative de connexion : 1
Tentative de connexion : 2
...
```

Supposons que la connexion réussisse à la troisième tentative. Lorsque la connexion réussit, afficher :

```
Console
Connexion réussie !
```

et les tentatives restantes ne doivent pas être exécutées.

**À utiliser :**

```
for
if
break
```

## 35. Exercice - Compte à rebours

Créer un compte à rebours :

```
Console
10
9
8
7
6
5
4
3
2
1
```

puis afficher :

```
Console
Application prête !
```

Essayer de résoudre le problème avec :

```
stride(...)
```

## 36. À retenir

Les trois principales boucles Swift sont :

```
for-in
while
repeat-while
```

```swift
for numero in 1...5 {
    print(numero)
}
```

for-in — à privilégier lorsque l'on parcourt une séquence ou lorsque le nombre d'itérations est connu.

```swift
while condition {
    // ...
}
```

while — à utiliser lorsque la répétition dépend principalement d'une condition.

```swift
repeat {
    // ...
} while condition
```

repeat-while — le bloc est exécuté au moins une fois.

Pour contrôler une boucle :

| Instruction | Effet |
|---|---|
| `break` | Arrête complètement la boucle |
| `continue` | Ignore l'itération actuelle |

## 37. Vision globale

Les notions vues jusqu'ici commencent maintenant à fonctionner ensemble :

```
Variables et constantes
        ↓
      Types
        ↓
    Booléens
        ↓
   Opérateurs
        ↓
   Conditions
        ↓
     Boucles
```

On peut désormais écrire des programmes qui :

- Mémorisent des informations
- Effectuent des calculs
- Prennent des décisions
- Répètent automatiquement des traitements